

CocoLuxe Punta Cana

SOLUCIÓN TÉCNICA: LIVE TRACKING + ENCUESTA POST-VIAJE

Sin app · Sin HyperTrack · Solo navegador del cliente · Supabase Realtime · Make · Google Maps
Caribbean Matrix SRL · Para el Desarrollador · Abril 2026

TECNOLOGÍA	ENCUESTA	COSTO ADICIONAL
navigator.geolocation Supabas e Realtime Google Maps API	Make (trigger) Resend (email+SMS)/feedback/[id] (página propia)	\$0 (todo en el stack existente)

1. Clarificación — Por Qué No se Usa HyperTrack para el Cliente

HyperTrack está diseñado para rastrear vehículos o choferes con su SDK instalado como app nativa. Para ese uso sí requiere app instalada. Sin embargo, lo que CocoLuxe necesita es diferente: el CLIENTE comparte su propia ubicación desde el navegador de su teléfono, sin instalar absolutamente nada. Para eso existe una solución nativa del navegador que no requiere ningún servicio externo adicional.

	HYPERTRACK	SOLUCIÓN COCOLUXE
Quién transmite	El chofer o el vehículo	El cliente desde su teléfono
Requiere app	Sí — app nativa iOS/Android	No — solo el navegador web
Costo	\$0 Free hasta 1,000 trips/mes	\$0 — incluido en Supabase y Google Maps
Implementación	SDK HyperTrack en app nativa	navigator.geolocation + Supabase Realtime
Control	Del operador o del chofer	100% del cliente — lo activa si quiere
Para CocoLuxe	Fase 2 (rastrear flota propia)	Fase 1 — experiencia del cliente

DECISIÓN: *HyperTrack se reserva para la Fase 2 cuando CocoLuxe quiera rastrear su propia flota con dispositivos GPS en los vehículos. Para la experiencia del cliente en Fase 1, la solución es navigator.geolocation + Supabase Realtime. Costo adicional: cero.*

2. Flujo Completo — De la Reserva a la Encuesta

Este es el flujo exacto de principio a fin. El cliente no necesita instalar nada. Todo ocurre en el navegador de su teléfono.



PASO 1 — Confirmación de reserva

Cuando el cliente completa la reserva, Make envía automáticamente un email y SMS con el enlace único de su reserva: cocoluxepuntacana.com/reserva/CL-48291. Este enlace es válido por 30 días y contiene todos los datos del servicio.



PASO 2 — Página de la reserva del cliente

El cliente abre el enlace en su navegador (móvil o desktop). Ve los datos de su servicio: fecha, hora, origen, destino, tipo de vehículo asignado y la placa. En el centro de la página hay un botón "Iniciar mi viaje" que está visible pero inactivo.



PASO 3 — El cliente decide activar el tracking

Esta parte es 100% decisión del cliente. Si no quiere activarlo, no lo activa y no pasa nada. Si toca "Iniciar mi viaje", el navegador le pregunta: "¿Permitir que cocoluxepuntacana.com acceda a tu ubicación?" — igual que cuando Google Maps pide permiso. El cliente acepta.



PASO 4 — Transmisión en tiempo real

navigator.geolocation del navegador empieza a enviar las coordenadas del cliente cada 5 segundos. Supabase Realtime las recibe y las distribuye instantáneamente. El cliente ve su posición moviéndose en un mapa de Google Maps integrado en la misma página.



PASO 5 — Compartir el viaje

El botón "Compartir viaje" genera un enlace de solo lectura: cocoluxepuntacana.com/seguimiento/CL-48291-live. Quien recibe ese enlace ve el mapa en tiempo real sin necesitar ningún acceso o login. El cliente lo manda a su familia, al hotel que lo espera, a quien quiera.



PASO 6 — Fin del viaje

El cliente toca "Llegué" o el sistema detecta automáticamente que el cliente está cerca del destino (radio de 200 metros). La transmisión se detiene. El mapa muestra la posición final. Supabase registra la hora de llegada y dispara el webhook a Make.



PASO 7 — Make espera 2 horas

Make recibe el evento de viaje completado desde Supabase. Activa un módulo de espera de exactamente 120 minutos. El cliente ya llegó, se instaló, tiene el servicio fresco en la memoria pero no está en el stress del viaje.



PASO 8 — Envío de la encuesta

Make activa Resend que envía simultáneamente un email y un SMS al cliente. El mensaje contiene tres opciones claramente diferenciadas: botón de Google Reviews, botón de TripAdvisor y enlace a la encuesta interna de CocoLuxe. Todo es opcional.



PASO 9 — Respuesta guardada automáticamente

Si el cliente responde la encuesta interna, Make recibe los datos via webhook desde `/api/feedback` y los guarda en Supabase tabla `feedback`. Si el rating es 1 o 2, Make envía una alerta inmediata a Kelmin. Si es 4 o 5, HubSpot actualiza el perfil del cliente.

3. Implementación Técnica — Código Completo

Todo el código va en el proyecto Next.js existente. No se instala ningún servicio nuevo. Se usan Supabase, Google Maps y Make que ya están en el stack.

3.1 — Nueva tabla en Supabase: live_tracking

Crear esta tabla en Supabase SQL Editor antes de escribir cualquier código:

```
CREATE TABLE live_tracking (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  booking_id TEXT REFERENCES bookings(id),  
  latitude DECIMAL(10, 8) NOT NULL,  
  longitude DECIMAL(11, 8) NOT NULL,  
  accuracy DECIMAL(8, 2),  
  speed DECIMAL(6, 2),  
  is_active BOOLEAN DEFAULT TRUE,  
  started_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  ended_at TIMESTAMP WITH TIME ZONE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);  
  
-- Activar Realtime en esta tabla  
ALTER TABLE live_tracking REPLICA IDENTITY FULL;  
  
-- RLS: Solo el backend puede escribir, el enlace público puede leer  
ALTER TABLE live_tracking ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "Public read for active tracking"  
  ON live_tracking FOR SELECT  
  USING (is_active = TRUE);
```

3.2 — Variables de Entorno a Agregar

Agregar en Vercel → Settings → Environment Variables y también en .env.local:

```
NEXT_PUBLIC_GOOGLE_MAPS_KEY=tu_google_maps_api_key  
MAKE_WEBHOOK_TRIP_COMPLETED=url_del_webhook_de_make
```

GOOGLE MAPS API KEY: *Obtener en console.cloud.google.com → APIs & Services → Credentials → Create API Key. Habilitar: Maps JavaScript API y Maps Embed API. Restringir la key solo al dominio cocoluxepuntacana.com para seguridad.*

3.3 — Página de la Reserva del Cliente

Crear el archivo: src/app/[locale]/reserva/[id]/page.jsx

```
'use client';  
import { useState, useEffect, useRef } from 'react';  
import { createClient } from '@supabase/supabase-js';  
  
const supabase = createClient(  

```

```

process.env.NEXT_PUBLIC_SUPABASE_URL,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
);

export default function BookingPage({ params }) {
  const { id } = params;
  const [booking, setBooking] = useState(null);
  const [tracking, setTracking] = useState(false);
  const [position, setPosition] = useState(null);
  const [shareUrl, setShareUrl] = useState(null);
  const [ended, setEnded] = useState(false);
  const watchIdRef = useRef(null);
  const trackingIdRef = useRef(null);

  // Cargar datos de la reserva al abrir la página
  useEffect(() => {
    async function loadBooking() {
      const { data } = await supabase
        .from('bookings')
        .select('*', vehicles('*'))
        .eq('id', id)
        .single();
      setBooking(data);
    }
    loadBooking();
  }, [id]);

  // Iniciar el tracking
  const startTracking = async () => {
    if (!navigator.geolocation) {
      alert('Tu navegador no soporta geolocalización');
      return;
    }

    // Crear registro de tracking en Supabase
    const { data: trackData } = await supabase
      .from('live_tracking')
      .insert({ booking_id: id, latitude: 0, longitude: 0 })
      .select()
      .single();

    trackingIdRef.current = trackData.id;
    setShareUrl(window.location.origin + '/seguimiento/' + id);
    setTracking(true);

    // Escuchar actualizaciones de posición cada 5 segundos
    watchIdRef.current = navigator.geolocation.watchPosition(
      async (pos) => {
        const coords = {
          latitude: pos.coords.latitude,
          longitude: pos.coords.longitude,
          accuracy: pos.coords.accuracy,
          speed: pos.coords.speed
        };
        setPosition(coords);
      }
    );
  };

```

```

    // Actualizar en Supabase - Realtime lo distribuye automáticamente
    await supabase
      .from('live_tracking')
      .update(coords)
      .eq('id', trackingIdRef.current);
  },
  (error) => console.error('GPS error:', error),
  { enableHighAccuracy: true, maximumAge: 5000, timeout: 10000 }
);
};

// Terminar el viaje
const endTrip = async () => {
  // Detener la escucha del GPS
  if (watchIdRef.current) {
    navigator.geolocation.clearWatch(watchIdRef.current);
  }

  // Marcar como inactivo en Supabase
  await supabase
    .from('live_tracking')
    .update({ is_active: false, ended_at: new Date().toISOString() })
    .eq('id', trackingIdRef.current);

  // Notificar a Make que el viaje terminó → Make espera 2h → encuesta
  await fetch('/api/trips/complete', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ bookingId: id })
  });

  setEnded(true);
  setTracking(false);
};

// Compartir el enlace
const shareJourney = () => {
  if (navigator.share) {
    navigator.share({
      title: 'My CocoLuxe Journey',
      text: 'Track my transfer in real time',
      url: shareUrl
    });
  } else {
    navigator.clipboard.writeText(shareUrl);
    alert('Link copied!');
  }
};

// RENDER - ver sección 3.5 para el diseño completo
return (
  <main style={{ background: '#0A0A0A', minHeight: '100vh', color: '#fff' }}>
    {/* Contenido de la página - ver sección 3.5 */}
  </main>

```

```
    );  
  }  
}
```

3.4 — Página de Seguimiento de Solo Lectura

Crear el archivo: `src/app/[locale]/seguimiento/[id]/page.jsx`

Esta página la ve quien recibe el enlace compartido. No tiene controles — solo el mapa.

```
'use client';  
import { useEffect, useState } from 'react';  
import { createClient } from '@supabase/supabase-js';  
  
const supabase = createClient(  
  process.env.NEXT_PUBLIC_SUPABASE_URL,  
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY  
);  
  
export default function TrackingView({ params }) {  
  const { id } = params;  
  const [position, setPosition] = useState(null);  
  const [booking, setBooking] = useState(null);  
  const [active, setActive] = useState(true);  
  
  useEffect(() => {  
    // Cargar datos de la reserva  
    supabase.from('bookings').select('*').eq('id', id)  
      .single().then(({ data }) => setBooking(data));  
  
    // Suscribirse a actualizaciones de posición en tiempo real  
    const channel = supabase  
      .channel('tracking-' + id)  
      .on('postgres_changes', {  
        event: 'UPDATE',  
        schema: 'public',  
        table: 'live_tracking',  
        filter: 'booking_id=eq.' + id  
      }, (payload) => {  
        if (!payload.new.is_active) {  
          setActive(false);  
          return;  
        }  
        setPosition({  
          lat: payload.new.latitude,  
          lng: payload.new.longitude  
        });  
      })  
      .subscribe();  
  
    return () => supabase.removeChannel(channel);  
  }, [id]);  
  
  // Ver sección 3.5 para el mapa de Google Maps  
  return (  
    <main style={{ background: '#0A0A0A', minHeight: '100vh' }}>
```

```

    {active ? (
      <div>Mapa en tiempo real – ver sección 3.5</div>
    ) : (
      <div>El viaje ha terminado. ¡Bienvenido!</div>
    )}
  </main>
);
}

```

3.5 — Integrar Google Maps en Ambas Páginas

1

Instalar librería

npm install @react-google-maps/api

2

Componente del mapa

Crear src/components/LiveMap.jsx

```

import { GoogleMap, Marker, useJsApiLoader } from '@react-google-maps/api';

export default function LiveMap({ position, destination }) {
  const { isLoading } = useJsApiLoader({
    googleMapsApiKey: process.env.NEXT_PUBLIC_GOOGLE_MAPS_KEY
  });

  if (!isLoading || !position) return <div>Esperando ubicación...</div>;

  return (
    <GoogleMap
      mapContainerStyle={{ width: '100%', height: '400px', borderRadius: '12px' }}
      center={position}
      zoom={14}
      options={{
        styles: darkMapStyle, // ver estilos negro/dorado abajo
        disableDefaultUI: true,
        zoomControl: true
      }}
    >
      <Marker
        position={position}
        icon={{
          url: '/images/marker-gold.png', // ícono dorado de CocoLuxe
          scaledSize: { width: 40, height: 40 }
        }}
      />
    </GoogleMap>
  );
}

// Estilos del mapa en paleta negro/dorado de CocoLuxe
const darkMapStyle = [
  { elementType: 'geometry', stylers: [{ color: '#0d1b2a' }] },
  { elementType: 'labels.text.fill', stylers: [{ color: '#C9A84C' }] },
  { featureType: 'road', elementType: 'geometry', stylers: [{ color: '#1a2a3a' }] },

```

```
{ featureType: 'water', elementType: 'geometry', stylers: [{ color: '#0a0a0a' }] },  
{ featureType: 'poi', stylers: [{ visibility: 'off' }] }  
];
```

3.6 — Endpoint que Notifica a Make al Terminar el Viaje

Crear: src/app/api/trips/complete/route.js

```
export async function POST(request) {  
  const { bookingId } = await request.json();  
  
  // Notificar a Make — Make espera 2h y envía la encuesta  
  await fetch(process.env.MAKE_WEBHOOK_TRIP_COMPLETED, {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({  
      booking_id: bookingId,  
      completed_at: new Date().toISOString(),  
      event: 'trip_completed'  
    })  
  });  
  
  return Response.json({ success: true });  
}
```

4. Diseño de la Página del Cliente — Lo Que Debe Ver

La página /reserva/[id] es lo primero que ve el cliente. Debe verse premium en cualquier teléfono. Misma paleta negro y dorado del sitio.

4.1 — Estructura Visual de la Página

SECCIÓN	CONTENIDO	ESTILO
Header	Logo CocoLuxe horizontal blanco sobre negro	height: 60px, background: #0A0A0A
Número de confirmación	CL-48291 en grande, dorado, centrado	font: Cormorant Garamond, size: 48px, color: #C9A84C
Tarjeta del servicio	Tipo, fecha, hora, origen, destino	background: #0D1B2A, border: 1px solid #C9A84C, border-radius: 12px
Tarjeta del vehículo	Nombre del vehículo y placa (cuando esté asignado)	Mismo estilo que tarjeta del servicio
Botón principal	"Start My Journey" / "Iniciar mi viaje"	background: #C9A84C, color: #0A0A0A, bold, border-radius: 8px, padding: 16px
Mapa en vivo	Google Maps con el punto dorado moviéndose	Aparece solo después de activar el tracking
Estado del viaje	Waiting / Active / Arrived — con indicador de color	Verde = activo, Gris = esperando, Dorado = llegó
Botón compartir	"Share Journey" con ícono de compartir	Aparece solo cuando el tracking está activo
Footer	CocoLuxe Punta Cana · info@cocoluxepuntacana.com	background: #0A0A0A, texto en gris claro

4.2 — Estados de la Página

ESTADO	CUÁNDO	BOTÓN VISIBLE	MAPA VISIBLE
Waiting	Al abrir el enlace — antes de activar	☒ "Iniciar mi viaje" — activo	☒ No — esperando activación del cliente
GPS Permission	El navegador preguntó — cliente no ha respondido	☐ Esperando permiso	☒ No
Active	Cliente aceptó y el GPS está transmitiendo	☒ "Llegué" — para terminar el viaje	☒ Sí — con el punto moviéndose
Sharing	Cliente tocó Compartir	☒ "Copiar enlace" o share nativo	☒ Sí
Ended	Cliente tocó Llegué o pasó el tiempo del servicio	☒ Ninguno	☒ Muestra posición final estática

⚠ El botón "Iniciar mi viaje" no debe activarse automáticamente. El cliente siempre debe tocarlo conscientemente. CocoLuxe nunca rastrea al cliente sin su consentimiento explícito.

5. Make — El Flujo de la Encuesta Post-Viaje

Este flujo en Make es el que conecta el fin del viaje con la encuesta. Se configura como un escenario separado en Make.

5.1 — Configurar el Webhook en Make

1	Nuevo escenario	Make → Create a new scenario → buscar "Webhooks" → Custom webhook
2	Copiar URL	Make genera una URL única → copiarla en la variable MAKE_WEBHOOK_TRIP_COMPLETED de Vercel
3	Módulo de espera	Después del webhook añadir: Tools → Sleep → Duration: 7200 seconds (2 horas)
4	Obtener datos de la reserva	Supabase → Select a Row → tabla bookings → filtrar por booking_id del webhook
5	Enviar email	Resend → Send Email → con los botones de Google, TripAdvisor y encuesta interna
6	Enviar SMS	Twilio o similar → Send SMS → mismo contenido en formato texto plano
7	Activar	Toggle en ON — el flujo corre automáticamente cada vez que llega el webhook

5.2 — El Email de la Encuesta — Contenido Exacto

El email que recibe el cliente 2 horas después de llegar. En el idioma en que reservó.

CocoLuxe Punta Cana
Thank you for traveling with us

We hope your experience was exceptional. If you have a moment, sharing your thoughts helps other travelers find us — and helps us keep improving.

★ **Leave a Google Review**
● **Review on TripAdvisor**

📄 Quick internal survey (3 questions)
All optional. Thank you for choosing CocoLuxe.

5.3 — Flujo Make Completo

MÓDULO EN MAKE	TIPO	CONFIGURACIÓN	RESULTADO
Trigger	Custom Webhook	URL guardada en MAKE_WEBHOOK_TRIP_COMPLETED	Recibe: booking_id, completed_at
Espera	Sleep	Duration: 7200 seconds	Pausa 2 horas antes de continuar
Datos reserva	Supabase → Select	tabla bookings, filtro id = booking_id	Obtiene: email, phone, lang, client name
Datos cliente	Supabase → Select	tabla clients, filtro booking → client_id	Obtiene: nombre completo del cliente
Email	Resend → Send Email	Template según idioma (lang del booking)	Email con 3 botones al cliente
SMS	Twilio → Send SMS	Mensaje corto con el link de encuesta	SMS al teléfono del cliente
HubSpot	HubSpot → Update Contact	Añadir propiedad: last_survey_sent = fecha	CRM actualizado automáticamente

6. Página de Encuesta Interna — /feedback/[id]

Esta página la construye el desarrollador directamente en Next.js. No se necesita Typeform ni ningún servicio externo. Vive en el dominio de CocoLuxe con los colores de la marca.

6.1 — Las 3 Preguntas

PREGUNTA	TIPO DE INPUT	OBLIGATORIA	QUÉ SE GUARDA
¿Cómo calificarías tu experiencia?	5 estrellas — botones visuales	Sí — no se puede enviar sin esto	rating (INTEGER 1-5)
¿Qué fue lo mejor del servicio?	Textarea libre — máximo 500 caracteres	No — opcional	comment (TEXT)
¿Algo que podríamos mejorar?	Textarea libre — máximo 500 caracteres	No — opcional	improvement (TEXT)

6.2 — Endpoint que Recibe la Encuesta

Crear: src/app/api/feedback/route.js

```
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY // Service role - acceso completo
);

export async function POST(request) {
  const { bookingId, rating, comment, improvement } = await request.json();

  // 1. Obtener el client_id de la reserva
  const { data: booking } = await supabase
    .from('bookings')
    .select('client_id')
    .eq('id', bookingId)
    .single();

  // 2. Guardar el feedback en Supabase
  await supabase.from('feedback').insert({
    booking_id: bookingId,
    client_id: booking.client_id,
    rating,
    comment,
    improvement,
    source: 'internal'
  });

  // 3. Notificar a Make - Make actualiza HubSpot y alerta si rating bajo
```

```

    await fetch(process.env.MAKE_WEBHOOK_FEEDBACK, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ bookingId, rating, comment })
    });

    return Response.json({ success: true });
  }
}

```

6.3 — Qué Hace Make con la Respuesta

CONDICIÓN	ACCIÓN DE MAKE
rating = 1 o 2 (experiencia mala)	Alerta inmediata a Kelmin por email y WhatsApp con todos los detalles — acción personal necesaria
rating = 3 (experiencia neutral)	Guardar en Supabase, actualizar HubSpot — sin acción inmediata
rating = 4 o 5 (experiencia buena)	Guardar en Supabase, actualizar HubSpot, enviar mensaje de agradecimiento al cliente con sugerencia de dejar review en Google
Sin respuesta a la encuesta	Después de 72 horas Make verifica — si no respondió, no se manda recordatorio (no incomodar al cliente)

7. Checklist de Implementación — En Orden

Verificar cada punto antes de pasar al siguiente. Si algo no funciona, no continúes hasta resolverlo.

Fase A — Base de Datos

1. Crear tabla `live_tracking` en Supabase con el SQL de la sección 3.1
2. Verificar que Realtime está activado en la tabla `live_tracking`
3. Verificar que RLS permite lectura pública solo cuando `is_active = TRUE`
4. Prueba: insertar un registro de prueba y verificar que aparece en Supabase

Fase B — Variables de Entorno

5. Agregar `NEXT_PUBLIC_GOOGLE_MAPS_KEY` en Vercel y en `.env.local`
6. Agregar `MAKE_WEBHOOK_TRIP_COMPLETED` (crear el webhook en Make primero)
7. Agregar `MAKE_WEBHOOK_FEEDBACK` (crear el webhook en Make primero)
8. Verificar que `SUPABASE_SERVICE_ROLE_KEY` está en las variables del servidor

Fase C — Páginas y Endpoints

9. Crear `src/app/[locale]/reserva/[id]/page.jsx` con el código de la sección 3.3
10. Crear `src/app/[locale]/seguimiento/[id]/page.jsx` con el código de la sección 3.4
11. Crear `src/components/LiveMap.jsx` con Google Maps y los estilos negro/dorado
12. Crear `src/app/api/trips/complete/route.js` con el código de la sección 3.6
13. Crear `src/app/[locale]/feedback/[id]/page.jsx` con las 3 preguntas
14. Crear `src/app/api/feedback/route.js` con el código de la sección 6.2

Fase D — Make y Flujos

15. Crear el flujo de encuesta en Make — Webhook → Sleep 7200s → Supabase → Resend → Twilio
16. Crear el flujo de feedback en Make — Webhook → Supabase → HubSpot → Alerta si rating bajo
17. Copiar las URLs de los webhooks en las variables de entorno de Vercel

Fase E — Pruebas Completas

18. Abrir `/reserva/[id]` en el teléfono móvil — verificar que carga correctamente
19. Tocar "Iniciar mi viaje" — el navegador debe pedir permiso de ubicación
20. Aceptar el permiso — el mapa debe aparecer con un punto dorado en tu posición
21. Abrir el enlace de seguimiento en otro teléfono — debe ver el punto moviéndose
22. Tocar "Llegué" — el mapa debe congelarse y Make debe recibir el webhook
23. Esperar 2 horas (o configurar el Sleep en 1 minuto para probar) — debe llegar el email de encuesta
24. Responder la encuesta con rating 2 — Kelmin debe recibir alerta inmediata
25. Responder la encuesta con rating 5 — HubSpot debe actualizarse

✓ Para probar el flujo completo sin esperar 2 horas reales, cambiar temporalmente el módulo Sleep de Make a 60 segundos. Volver a 7200 segundos antes de lanzar en producción.

8. Resumen — La Solución Completa

TRACKING DEL CLIENTE navigator.geolocation↓Supabase live_tracking↓Supabase Realtime↓Google Maps en el navegador SIN APP · SIN HYPERTRACKSIN COSTO ADICIONAL	ENCUESTA POST-VIAJE /api/trips/complete↓Make webhook↓Sleep 2 horas↓Resend email + SMS↓3 opciones al cliente AUTOMÁTICO · SIN INTERVENCIÓN HUMANA	LO QUE VE EL CLIENTE Enlace de reserva↓Botón "Iniciar mi viaje"↓Mapa dorado en vivo↓Botón compartir↓2h después: encuesta opcional DECISIÓN 100% DEL CLIENTE
--	---	--

PARA KELMIN: Cuando el desarrollador tenga las pruebas funcionando, hacer una reserva real de \$1 entre dos teléfonos para verificar que el cliente ve el mapa, el familiar ve el seguimiento en el segundo teléfono y el email de encuesta llega 2 horas después. Esa es la prueba de aceptación final.